

Python on Your Phone

A practical class guide to writing and running Python on Android, using Acode as the editor and Termux as the terminal.

PREPARED BY JOHN CHIBUZOR JOHN · ANDROID 7.0+ · NO LAPTOP REQUIRED

```
~/PythonClass $ python hello.py
What is your name? Chidinma
Hello, Chidinma
Welcome to Python on Android.
~/PythonClass $ _
```

CONTENTS

01 What We Are Building	08 Termux Survival Skills
02 Understanding the Two Apps	09 Installing Packages
03 Installation	10 Troubleshooting Table
04 First-Time Termux Setup	11 Suggested 5-Day Lesson Plan
05 The Shared Folder — The Concept Students Must Not Skip	12 Classroom Exercises
06 Setting Up Acode	13 Tutor Checklist Before Class
07 The Core Workflow — Write, Save, Run	14 Student Quick Reference Card

01 What We Are Building

By the end of this material, every student will be able to:

1. Install and set up Termux and Acode correctly.
2. Write a Python file in Acode and run it in Termux.
3. Understand the *edit* → *save* → *run* → *fix* loop.
4. Install and use Python packages with `pip`.
5. Debug the common phone-specific errors that frustrate beginners.

Requirements: an Android phone (Android 7.0 or newer), about 1 GB free storage, and internet for the first setup.

iPhone students: Termux is Android-only. Point them to *Pythonista* or *a-Shell* instead — the Python lessons from Section 7 onward still apply.

02 Understanding the Two Apps

Beginners get confused because they expect one app to do everything. Teach this distinction clearly on Day 1:

	ACODE	TERMUX
What it is	A code editor	A Linux terminal
What it does	Where you <i>write</i> code	Where you <i>run</i> code
Real-world equal	VS Code / Notepad+ +	Command Prompt / Bash
Can it run Python?	Not by itself	Yes
Can it write code nicely?	Yes — colours, indentation, autocomplete	Painful

The one-line summary for students:

"Acode is the pen. Termux is the machine that reads what you wrote."

This mirrors real development: professionals also write in one tool and run in another.

03 Installation

3.1 Termux — where to install it from

Termux exists in two builds, and this trips up more beginners than any Python concept will.

The recommended route is F-Droid. The official release sources for Termux are F-Droid and GitHub, and the Google Play Store releases remain deprecated. The Play Store version was set aside because of policy changes introduced with Android 10 that restricted apps from executing files in their internal data directory.

Steps (F-Droid):

1. Open the phone browser and go to `https://f-droid.org`
2. Download and install the F-Droid app (allow "Install unknown apps" for the browser when prompted).
3. Open F-Droid → search **Termux** → Install.
4. *Alternative:* download the Termux APK directly from the F-Droid Termux page, or from GitHub Releases.

If Play Protect blocks installation: Play Store → profile icon → Play Protect → temporarily turn off scanning, install, then turn it back on.

THE PLAY STORE BUILD – WHEN IT IS ACCEPTABLE

Some students will already have Termux from the Play Store, or their device or school policy may not permit sideloading. That is workable, with conditions.

A Play Store release (v0.120) was published in 2024 for Android 11 and newer devices, using a workaround for the Android 10 exec restrictions. It installs packages and runs Python normally. The trade-off: it has the changes from ten app releases reverted, making it functionally equivalent to the much older v0.108, so features, bug fixes and security fixes from those releases are missing, and the app has reduced permissions.

Older Play Store installs (from 2021 or before) are a different matter — their package repositories are dead and `pkg update` will fail outright.

Have every student run this check on Day 1:

TERMUX

```
termux-info
pkg update
```

`termux-info` prints the app version and the repository in use. If `pkg update` finishes without 403 or 404 errors, that phone is fine for the whole course, whichever store it came from.

RULES FOR A MIXED CLASSROOM

- **Pick one route for the class if you can.** Debugging two environments at once during a live session is what actually costs you time — not the build itself.
- Students on a **working** Play Store build should simply continue. Do not make them reinstall for the sake of it.
- Students on a **dead** Play Store build (failing `pkg update`) must move to F-Droid.
- If AcodeX, Node.js or anything heavier misbehaves, the Play Store build is the first suspect. Move that student to F-Droid rather than debugging further.

SWITCHING FROM PLAY STORE TO F-DROID

You cannot switch by updating. The two builds are signed with different keys, so the Play Store app and all its plugin apps (Termux:API, Termux:Styling) must be **uninstalled first**, which wipes the Termux filesystem.

Back up anything valuable to shared storage before uninstalling:

TERMUX

```
termux-setup-storage
tar -czf ~/storage/shared/termux-backup.tar.gz -C /data/data/com.termux/files ./home
```

Also turn off auto-update for Termux in the Play Store afterwards, so it does not keep trying to reinstall over the F-Droid version.

3.2 Acode

Acode is on the Google Play Store — install it normally. Search "**Acode - code editor | FOSS**".

04 First-Time Termux Setup

Open Termux. Type each line and press Enter. **The first command takes several minutes — warn the class in advance so nobody force-**

closes it.

TERMUX

```
pkg update && pkg upgrade -y
```

Updates the package lists and installed packages. If asked about configuration files, press Enter to accept the default.

TERMUX

```
pkg install python -y
```

Installs Python 3 and `pip`.

TERMUX

```
python --version
```

Should print something like `Python 3.12.x`. If this fails, stop and fix it before continuing — nothing else will work.

TERMUX

```
termux-setup-storage
```

Android will pop up a permission dialog. **Tap Allow**. This creates a folder called `storage` inside Termux that links to the phone's normal file storage.

05 The Shared Folder — The Concept Students Must Not Skip

Termux has its own private home folder. **Acode cannot see inside it**. If a student saves work in Termux's home folder, Acode will never find the file, and vice versa.

The solution: keep all class work in **shared storage**, which both apps can reach.

Create the class folder:

TERMUX

```
mkdir -p ~/storage/shared/PythonClass
cd ~/storage/shared/PythonClass
pwd
```

`pwd` should print:

REFERENCE

```
/data/data/com.termux/files/home/storage/shared/PythonClass
```

On the phone's file manager, that same folder appears as **Internal Storage → PythonClass**.

Teaching analogy: *Acode and Termux are two people who cannot enter each other's bedrooms. `PythonClass` is the sitting room where they both drop their books.*

06 Setting Up Acode

1. Open Acode.
2. Tap the **folder icon** (side menu) → **Open folder**.
3. Navigate to **Internal Storage → PythonClass → Use this folder / Allow**.
4. The folder now stays in the sidebar permanently.

Settings to change before the first lesson

Go to **⋮ → Settings → Editor** and set:

SETTING	VALUE	WHY
Tab size	4	Python convention
Soft tabs (use spaces)	ON	Mixing tabs and spaces breaks Python
Auto indent	ON	Saves typing
Line numbers	ON	Error messages report line numbers

Word wrap	ON	Small screen
Auto save	ON (or teach saving discipline)	Students forget to save, then wonder why nothing changed

Critical: turn off keyboard autocorrect

Phone keyboards are the number one cause of mystery errors in mobile Python. They convert " into " and ' into ', and capitalise the first letter of every line — so `print` silently becomes `Print`.

Have every student go to **Android Settings → System → Languages & input → On-screen keyboard → Gboard → Text correction** and turn off: - Auto-capitalisation - Auto-correction - Smart punctuation / curly quotes

Better still, install **Hacker's Keyboard** or **Unexpected Keyboard** (F-Droid) — they give real arrow keys, a Tab key, and straight quotes.

07 The Core Workflow — Write, Save, Run

This is the loop the entire course is built on. Drill it until it is automatic.

Step 1 — Write in Acode. In the `PythonClass` folder, create a new file named `hello.py`:

```
ACODE — PYTHON

name = input("What is your name? ")
print("Hello,", name)
print("Welcome to Python on Android.")
```

Step 2 — Save. Tap the save icon. (Unsaved work runs as the *old* version — a classic beginner trap.)

Step 3 — Run in Termux. Switch to Termux and type:

```
TERMUX

cd ~/storage/shared/PythonClass
python hello.py
```

Step 4 — Fix and repeat. Change the code in Acode, save, switch back to Termux, press the **up arrow** (or Volume Up + W) to bring back the last command, and press Enter.

Making the app-switching bearable

Three options, in order of ease:

- 1. **Recent apps button** — tap twice to swap between Acode and Termux. Works everywhere, teach this first.
- 2. **Split screen** — long-press the app icon in Recents → Split screen → put Acode on top, Termux below. This is the best classroom setup.
- 3. **AcodeX — a terminal inside Acode.** Install the plugin from Acode → Settings → Plugins → AcodeX - Terminal, then install the server in Termux with `curl -sL https://raw.githubusercontent.com/bajrangCoder/acode-plugin-acodex/main/installServer.sh | bash` and start it by running `acodex-server` (short alias: `axs`). Termux must be running in the background for it to work.

Tutor advice: teach options 1 and 2 first. Introduce AcodeX only in week 2, once students are comfortable — an extra moving part early on creates support tickets, not learning.

08 Termux Survival Skills

Essential commands

COMMAND	MEANING
<code>pwd</code>	Where am I?
<code>ls</code>	List files here
<code>cd PythonClass</code>	Enter a folder
<code>cd ..</code>	Go back one folder
<code>cd ~</code>	Go to Termux home
<code>python file.py</code>	Run a Python file
<code>python</code>	Open the interactive shell (type <code>exit()</code> to leave)
<code>clear</code>	Clean the screen
<code>nano file.py</code>	Emergency editor inside Termux (Ctrl+O save, Ctrl+X exit)

Hardware key shortcuts

KEYS	ACTION
Volume Down + C	Ctrl+C — stop a running program
Volume Down + D	Exit Termux session
Volume Down + L	Clear screen
Volume Up + W / A / S / D	Arrow keys ↑ ← ↓ →
Volume Up + T	Tab (autocomplete file names)
Volume Up + Q	Show/hide the extra keys row

Teach `Volume Up + T` early. Typing long filenames on a phone is misery; Tab autocomplete removes most of it.

Other gestures

- **Long-press** the screen → copy, paste, and the Termux menu.
- **Swipe from the left edge** → open a new session (useful: one session running a program, one for commands).
- Pinch to zoom text size.

09 Installing Packages

```
TERMUX
pip install requests
pip list
```

Warn students: packages that contain compiled C code (numpy, pandas, matplotlib) often fail with plain `pip` on Android. Use Termux's own builds where they exist:

```
TERMUX
pkg install python-numpy
```

For the first weeks, stick to the standard library and pure-Python packages (`requests`, `rich`, `colorama`). Save data-science libraries for later, or use a laptop.

10 Troubleshooting Table

Print this and hand it out. It answers roughly 90% of student questions.

ERROR MESSAGE	REAL CAUSE	FIX
<code>python: command not found</code>	Python not installed	<code>pkg install python</code>
<code>can't open file 'hello.py': No such file or directory</code>	Wrong folder	<code>pwd</code> , then <code>cd ~/storage/shared/PythonClass</code> , then <code>ls</code>
<code>SyntaxError: invalid character ''</code>	Keyboard smart quotes	Turn off smart punctuation; retype the quotes
<code>NameError: name 'Print' is not defined</code>	Auto-capitalisation	Turn off auto-capitalise; Python is case-sensitive
<code>IndentationError</code> / <code>TabError</code>	Mixed tabs and spaces	Turn on soft tabs (4 spaces) in Acode; retype the indentation
Code changes do nothing	File not saved	Save in Acode before running
Acode cannot see the file	Saved in Termux home	Move it into <code>~/storage/shared/PythonClass</code>
<code>Permission denied</code> on storage	Storage permission not granted	Re-run <code>termux-setup-storage</code> and tap Allow
Termux dies while a program runs	Android killed the background app	Pull down the Termux notification → Acquire wakelock
<code>Unable to locate package</code>	Stale package lists	Run <code>pkg update</code> first, then retry the install
<code>pkg update</code> fails with 403 / 404	Old Play Store build with dead repositories	Back up, uninstall, reinstall from F-Droid (Section 3.1)

11 Suggested 5-Day Lesson Plan

Day 1 — Setup and Hello World. Install both apps, run `pkg update`, install Python, run `termux-setup-storage`, create `PythonClass`, open it in Acode, write and run `hello.py`. *Success = every student's phone prints their own name.* Budget the whole session for this — installations are

slow and someone's storage will be full.

Day 2 — Variables, input, and types. Build a `greeting.py` that collects name and age. Introduce `int()`, `str()`, f-strings. Exercise: a program that prints the year the user will turn 100.

Day 3 — Conditions and loops. `if / elif / else`, `for`, `while`. Exercise: a number-guessing game with `random`. This is where indentation errors appear — good, teach the fix live.

Day 4 — Functions and lists. `def`, arguments, return values, list methods. Exercise: a simple grade calculator that averages a list of scores.

Day 5 — A small project + packages. `pip install requests`, read from a file, write to a file. Mini project: a command-line to-do list saved to `todo.txt` that survives closing the app.

12 Classroom Exercises

1. **Setup proof.** Screenshot Termux showing `python --version` and `pwd` pointing at `PythonClass`.
2. **Calculator.** Take two numbers and an operator, print the result. Handle division by zero.
3. **Data prices.** Store Nigerian data bundle prices in a dictionary; let the user look one up by name.
4. **Airtime split.** A user has ₦5,000 to share among N people — print each person's share and the remainder.
5. **Debug drill.** Give students a file with three deliberate bugs (smart quote, wrong indentation, capital `Print`) and have them fix it using only the error messages.
6. **Final project.** A to-do list that adds, lists, and deletes tasks from a text file.

13 Tutor Checklist Before Class

- ☐ Ask students to install Termux and Acode **before** the session — the downloads eat class time and data.
- ☐ Decide and announce one install route (F-Droid or Play Store) so the class is on one environment.
- ☐ Have students run `termux-info` and `pkg update` before class and report failures to you in advance.
- ☐ Confirm each phone has at least 1 GB free.
- ☐ Prepare a hotspot or confirm campus Wi-Fi; `pkg update` on 30 phones is heavy.
- ☐ Have 2–3 spare phones ready for students whose devices fail.
- ☐ Test the whole flow yourself on the morning of class — Termux repositories occasionally change.
- ☐ Print the troubleshooting table (Section 10) for every student.
- ☐ Have a fallback: **Pydroid 3** (Play Store) is an all-in-one Python IDE if a student's device refuses Termux entirely.

14 Student Quick Reference Card

REFERENCE

```
SETUP (once)
  pkg update && pkg upgrade -y
  pkg install python -y
  termux-setup-storage
  mkdir -p ~/storage/shared/PythonClass

EVERY SESSION
  Termux: cd ~/storage/shared/PythonClass
  Acode:  write code, SAVE
  Termux: python myfile.py
  Repeat: Volume Up + W (last command) then Enter

GOLDEN RULES
  1. Save before you run.
  2. Keep all files in PythonClass.
  3. Autocorrect off. Always.
  4. Read the LAST line of the error first.
  5. Indent with 4 spaces, never tabs.
```

Prepared for classroom use. Verify the Termux install source each term — the project's distribution channels have changed before and may change again.